Soccer Analytics — Codebase Guide

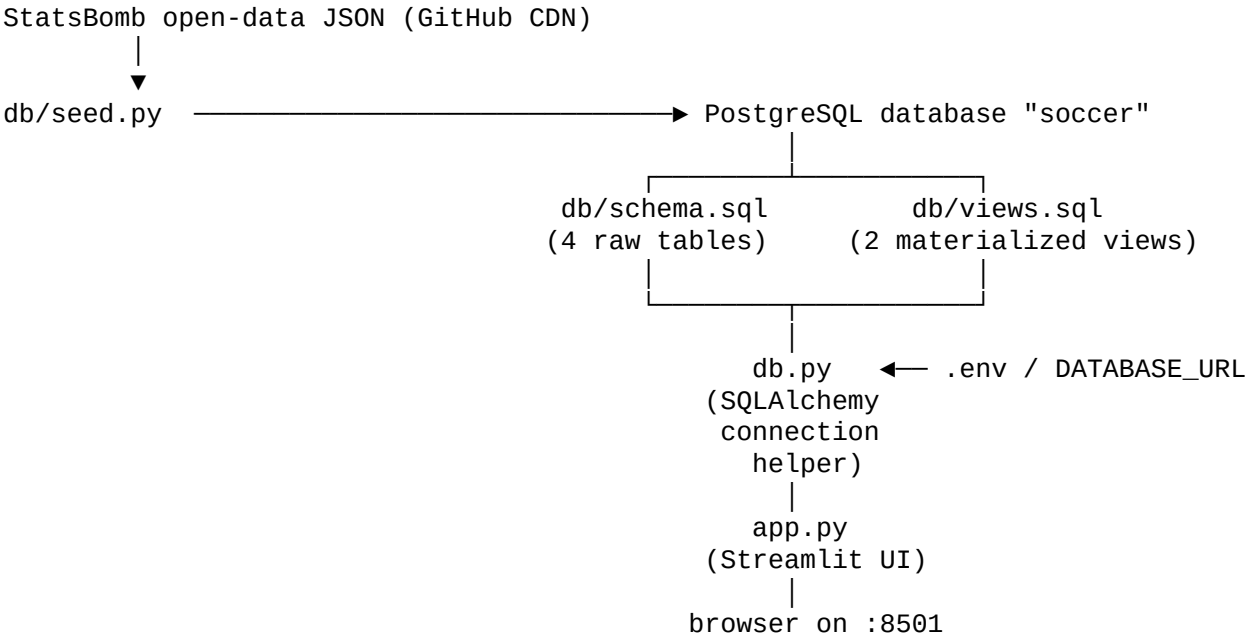
Overview

A web analytics dashboard built on top of StatsBomb's free open event data. The stack is:

- **PostgreSQL** — stores all raw data and precomputes summaries as materialized views
- **Python** — connects to the database and drives the web UI
- **Streamlit** — the web framework; the entire frontend is a single Python script
- **R (optional)** — a separate heatmap service that can generate goal-location visualisations; not required for the main dashboard

---

How the pieces fit together



Data flows in one direction: seed → database → views → Python → browser. Nothing writes back to the database at runtime.

File-by-file summary

File	Role
`db/schema.sql`	Defines the 4 raw tables and their indexes
`db/seed.py`	Downloads StatsBomb JSON from GitHub and populates the tables
`db/views.sql`	Creates the two materialized views that precompute stats and percentiles
`db.py`	SQLAlchemy connection pool; exposes a single `query()` function
`app.py`	The entire Streamlit application — layout, styling, data fetching, rendering

File	Role
`requirements.txt`	Python dependencies
`.streamlit/config.toml`	Streamlit theme (light mode, blue accent)
`.env.example`	Template for the DATABASE_URL environment variable

Shell commands

First-time setup

```
# 1. Create the database
createdb soccer

# 2. Create the 4 raw tables and indexes
psql -d soccer -f db/schema.sql

# 3. Seed data — pick one:

#    Python (pulls directly from StatsBomb's GitHub CDN, no R needed)
pip install -r requirements.txt
python db/seed.py

#    R (uses the StatsBombR package; slower but produces identical data)
Rscript db/seed.R

# 4. Build the materialized views (run after seeding, and again after any re-seed)
psql -d soccer -f db/views.sql
```

Running the app

```
streamlit run app.py
# → http://localhost:8501
```

Refreshing after a re-seed

The materialized views are snapshots. If you re-run the seeder you must rebuild them:

```
psql -d soccer -f db/views.sql
# (the file drops and recreates both views from scratch)
```

Or refresh in-place without dropping:

```
psql -d soccer -c "REFRESH MATERIALIZED VIEW top_players;"
psql -d soccer -c "REFRESH MATERIALIZED VIEW top_teams;"
```

Database schema

```
competitions  ──────────────────────────┐
  competition_id (PK, part 1)           │
  season_id      (PK, part 2)           │
```

```
  competition_name                    │
  season_name                         │
                                      │ FK
matches  ─────────────────────────────┤
  match_id (PK)                       │
  competition_id ─────────────────────┘
  season_id
  match_date, home_team, away_team
  home_score, away_score

players
  player_id (PK)
  player_name
  nationality

events  (3–5 million rows)
  event_id (UUID PK)
  match_id  → matches
  player_id → players
  team, type, minute, second
  x, y          (pitch coords, 0–120 × 0–80)
  outcome       (e.g. "Goal", "Saved", "Blocked")
  competition_id, season_id   (denormalised for fast filtering)
```

Indexes on `events`: `player_id`, `team`, `type`, `match_id` — without these the analytical queries would be extremely slow on 3–5 M rows.

Materialized views

Both views live in `db/views.sql` and are the only thing `app.py` reads at query time. They precompute all aggregations and percentiles so the web requests are fast.

`top_players` — top 100 players per competition/season by goals, with percentile ranks: - `total_goals`, `total_shots`, `total_passes`, `conversion_rate` - `goals_pct`, `shots_pct`, `passes_pct`, `conversion_pct` — each is `PERCENT_RANK()` relative to all players in that competition/season, expressed as 0–100

`top_teams` — all teams per competition/season, with the same percentile columns: - `goals_scored`, `total_shots`, `total_passes`, `shot_conversion` - `goals_pct`, `shots_pct`, `passes_pct`, `conversion_pct`

Line-by-line file explanations

`db/schema.sql`

`CREATE TABLE IF NOT EXISTS competitions (`

Creates the table only if it does not already exist, so this file is safe to re-run.

```
    competition_id INTEGER,
    season_id INTEGER,
```

StatsBomb's numeric IDs for each competition (e.g. 2 = Champions League) and each season (e.g. 27 = 2015/16).

```
    PRIMARY KEY (competition_id, season_id)
```

A composite primary key — neither ID alone is unique; the pair is.

```
CREATE TABLE IF NOT EXISTS matches (
    match_id INTEGER PRIMARY KEY,
    competition_id INTEGER,
    season_id INTEGER,
```

Each match belongs to exactly one competition/season.

```
   FOREIGN KEY (competition_id, season_id) REFERENCES competitions(competition_id, season_id)
```

Enforces that you cannot insert a match for a competition that does not exist in the competitions table.

```
CREATE TABLE IF NOT EXISTS players (
    player_id INTEGER PRIMARY KEY,
    player_name TEXT,
    nationality TEXT
```

One row per unique player across the entire dataset, regardless of which competition they appeared in.

```
CREATE TABLE IF NOT EXISTS events (
    event_id UUID PRIMARY KEY,
```

StatsBomb uses UUIDs for event IDs, not integers.

```
    type TEXT,
```

The event category: `'Shot'`, `'Pass'`, `'Carry'`, `'Dribble'`, etc. Everything the analytics is built on filters by this column.

```
    x NUMERIC,
    y NUMERIC,
```

Pitch coordinates in StatsBomb's system: x runs 0–120 from goal line to goal line, y runs 0–80 from touchline to touchline.

```
    outcome TEXT,
```

Only populated for shots: `'Goal'`, `'Saved'`, `'Blocked'`, `'Off T'`, `'Post'`, `'Wayward'`. NULL for non-shot events.

```
    competition_id INTEGER,
    season_id INTEGER
```

Deliberately denormalised — these could be derived via `match_id → matches`, but storing them directly on events makes the analytical queries much faster because you avoid joining through matches every time you filter by competition.

```
CREATE INDEX IF NOT EXISTS idx_events_player ON events(player_id);
CREATE INDEX IF NOT EXISTS idx_events_team   ON events(team);
CREATE INDEX IF NOT EXISTS idx_events_type   ON events(type);
CREATE INDEX IF NOT EXISTS idx_events_match  ON events(match_id);
```

Four indexes on the columns that appear in WHERE clauses. On a 3–5 M row table without these, every query would require a full sequential scan.

---

db/seed.py

`BASE_URL = "https://raw.githubusercontent.com/statsbomb/open-data/master/data"`

StatsBomb hosts their free data as raw JSON files on GitHub. All requests hit this base URL.

```
def fetch(url):
    r = requests.get(url, timeout=30)
    r.raise_for_status()
    return r.json()
```

A thin wrapper around `requests.get`. `raise_for_status()` turns HTTP 4xx/5xx responses into exceptions rather than silently returning empty data.

```
def extract_xy(location):
    if isinstance(location, list) and len(location) >= 2:
        return location[0], location[1]
    return None, None
```

StatsBomb stores coordinates as a JSON array `[x, y]`. This function safely unpacks it, returning `None, None` for events that have no location (e.g. substitutions, cards).

`cur.execute("TRUNCATE events, players, matches, competitions RESTART IDENTITY CASCADE")`

Wipes all four tables before re-loading. `CASCADE` handles the foreign-key order automatically. `RESTART IDENTITY` resets any sequences (though these tables use natural IDs not sequences).

`comps = fetch(f"{BASE_URL}/competitions.json")`

Downloads the master list of all StatsBomb free competitions — a flat JSON array with one entry per competition/season pair.

`psycopg2.extras.execute_values(cur, "INSERT INTO competitions ...", [...])`

Batches all competition rows into a single INSERT rather than one INSERT per row — much faster for large data.

```
for comp in comps:
    matches = fetch(f"{BASE_URL}/matches/{cid}/{sid}.json")
```

Iterates every competition/season and downloads its match list. The URL path uses StatsBomb's competition_id and season_id integers.

```
for m in matches:
    events = fetch(f"{BASE_URL}/events/{mid}.json")
```

Downloads the full event stream for each match individually. Each file can contain thousands of rows.

`e.get("shot", {}).get("outcome", {}).get("name") if e.get("shot") else None,`

Shot outcome is nested three levels deep in the JSON. If the event is not a shot, `e.get("shot")` returns None and the whole expression short-circuits to None.

```
if pid not in seen_players:
    seen_players.add(pid)
    player_rows.append(...)
```

Tracks which player IDs have already been staged in this run so that duplicate player rows are not accumulated before the INSERT.

`ON CONFLICT (player_id) DO NOTHING`

A player can appear across many matches; this silently ignores duplicate player inserts instead of raising an error.

db/views.sql

```
DROP MATERIALIZED VIEW IF EXISTS top_players;
DROP MATERIALIZED VIEW IF EXISTS top_teams;
```

Drops the old views before recreating them. The order matters: if `top_teams` depended on `top_players` you would need to drop the dependent first; here order is arbitrary but explicit.

```
CREATE MATERIALIZED VIEW top_players AS
WITH all_player_stats AS (
```

A materialized view stores the query result on disk like a table, so subsequent reads are instant rather than recomputing the aggregations every time. The `WITH` clause defines a CTE (common table expression) that is referenced below.

```
   COUNT(*) FILTER (WHERE e.type = 'Shot' AND e.outcome = 'Goal') AS total_goals,
```

`FILTER (WHERE ...)` is a PostgreSQL extension to aggregate functions. It counts only the rows matching the condition, equivalent to `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` but more readable.

```
    ROUND(
      COUNT(*) FILTER (WHERE e.type = 'Shot' AND e.outcome = 'Goal')::NUMERIC
      / NULLIF(COUNT(*) FILTER (WHERE e.type = 'Shot'), 0), 3
    ) AS conversion_rate
```

Divides goals by shots. `NULLIF(..., 0)` prevents a division-by-zero error by returning NULL when a player has zero shots. The `::NUMERIC` cast ensures the division is not integer division.

```
    ROW_NUMBER() OVER (
      PARTITION BY competition_id, season_id
      ORDER BY total_goals DESC, total_shots DESC
    ) AS rank,
```

Assigns each player a rank within their competition/season. `PARTITION BY` resets the counter for each competition/season pair. The tiebreaker is total shots when two players have the same number of goals.

```
    ROUND(PERCENT_RANK() OVER (
      PARTITION BY competition_id, season_id ORDER BY total_goals
    ) * 100) AS goals_pct,
```

`PERCENT_RANK()` returns a value between 0.0 and 1.0 representing where this player sits relative to all players in the same competition/season. Multiplied by 100 and rounded, this becomes a 0–100 percentile. The lowest-ranked player gets 0, the highest gets 100.

```
SELECT * FROM ranked WHERE rank <= 100;
```

Filters down to the top 100 players per competition/season. The percentile columns are computed across all players before this filter is applied, so a player ranked 100th still has a percentile relative to the full field.

```
CREATE MATERIALIZED VIEW top_teams AS
WITH all_team_stats AS (
  ...
)
SELECT *,
  ROW_NUMBER() OVER (...) AS rank,
  ROUND(PERCENT_RANK() OVER (...) * 100) AS goals_pct,
  ...
FROM all_team_stats;
```

Same structure as `top_players` but aggregated at the team level. All teams are included (no `WHERE rank <= N` filter) because there are far fewer teams than players per competition.

---

db.py

```
from dotenv import load_dotenv
load_dotenv()
```

Reads `.env` from the current directory and injects its contents into the process environment. This happens at import time, before anything else reads `os.getenv`.

```
_user = os.environ.get("USER", "simran")
DATABASE_URL = os.getenv("DATABASE_URL", f"postgresql://{_user}@/soccer?host=/var/run/postgr
```

Builds a default connection string for local PostgreSQL via Unix socket. The `host=/var/run/postgresql` parameter tells psycopg2 to connect via socket rather than TCP, which does not require a password on a typical Linux install. If `DATABASE_URL` is set in the environment (or `.env`), that takes precedence.

```
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)
```

Creates a SQLAlchemy engine (a connection pool) once at module load time. `sessionmaker` is a factory — calling `SessionLocal()` produces a new session from the pool.

```
def get_db():
    return SessionLocal()
```

Returns a new database session. The caller is responsible for closing it.

```
def query(sql, params=None):
    db = get_db()
    try:
        result = db.execute(text(sql), params or {})
        return [dict(r._mapping) for r in result.fetchall()]
    finally:
        db.close()
```

The one function the rest of the codebase uses. `text(sql)` wraps a raw SQL string so SQLAlchemy treats it as a literal query. `params or {}` avoids passing `None` when no parameters are given. `r._mapping` is SQLAlchemy's way to access a row as a dict-like object; converting it to a plain `dict` makes it serialisable and easier to work with. The `finally` block ensures the session is always closed even if an exception is raised.

---

app.py

```
st.set_page_config(
    page_title="Soccer Analytics",
    layout="wide",
    initial_sidebar_state="expanded",
)
```

Must be the first Streamlit call in the script. `layout="wide"` removes the default narrow centered column so the card grid can use the full browser width.

```
st.markdown("""<style> ... </style>""", unsafe_allow_html=True)
```

Injects a <style> block into the page. unsafe_allow_html=True is required because Streamlit sanitises HTML by default. All custom styling lives here.

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=sw
```

Loads the Inter typeface from Google Fonts. This is a network request made by the browser at page load; it will fail in offline environments.

```
.stApp { background: #f8fafc; }
```

Overrides Streamlit's default background on the main content area.

```
section[data-testid="stSidebar"] { background: #ffffff; border-right: 1px solid #e2e8f0; }
```

Targets the sidebar by its data-testid attribute — this is more stable than targeting by generated class names, which can change between Streamlit versions.

```
.stTabs [data-baseweb="tab-list"] { ... }
.stTabs [aria-selected="true"] { color: #2563eb !important; border-bottom: 2px solid #2563eb !
```

Streamlit's tab component is built on the BaseWeb library. Targeting [data-baseweb="tab"] and [aria-selected="true"] is the only reliable way to restyle tabs without !important fighting Streamlit's own specificity.

```
.stat-card { background: #ffffff; border: 1px solid #e2e8f0; border-
radius: 10px; ... }
.stat-card:hover { border-color: #2563eb; box-shadow: 0 2px 8px rgba(37,99,235,0.08); }
```

The card component used for every team and player. The hover state gives a subtle blue border and shadow to signal interactivity.

```
.stat-row { display: grid; grid-template-columns: 90px 38px 1fr 36px; ... }
```

Each metric row inside a card is a 4-column CSS grid: label (fixed 90px) | value (fixed 38px) | bar (flexible, takes remaining space) | percentile label (fixed 36px). This ensures all bars are the same width regardless of the label or value text.

```
.bar-low  { background: #f97316; }   /* orange  — below 33rd percentile */
.bar-mid  { background: #eab308; }   /* yellow  — 33rd to 60th          */
.bar-high { background: #22c55e; }   /* green   — 60th to 85th          */
.bar-elite{ background: #2563eb; }   /* blue    — above 85th            */
```

Four colour bands that give an immediate visual signal of performance tier.

```
def pct_bar(value: float, label: str, display: str) -> str:
```

Returns an HTML string for one stat row. value is the percentile (0–100), label is the row name shown on the left, display is the formatted metric value shown next to the label.

```
    pct = float(value or 0)
    width = max(2, int(pct))
```

Converts the database value to a float. The minimum bar width of 2px ensures even a 0th-percentile player shows a sliver of bar rather than nothing.

```
   suffix = "th" if 4 <= int(pct) % 100 <= 20 else {1: "st", 2: "nd", 3: "rd"}.get(int(pct) % 10,
```

Correct English ordinal suffix. The 4 <= n % 100 <= 20 rule handles the special cases 11th, 12th, 13th (which would otherwise get "st", "nd", "rd"). The dict lookup handles 1st, 2nd, 3rd for all other numbers ending in 1, 2, or 3.

```
def player_card(p: dict) -> str:
    conv = float(p.get("conversion_rate") or 0) * 100
```

Conversion rate is stored as a decimal (0.266) in the database; multiplying by 100 converts it to a percentage for display.

```
def team_card(t: dict) -> str:
```

Identical structure to `player_card` but uses team column names (`goals_scored`, `shot_conversion`) instead of player column names.

```
def summary_strip(items: list[tuple]) -> str:
    pills = "".join(
        f'<div class="summary-pill">...' for label, value in items
    )
    return f'<div class="summary-strip">{pills}</div>'
```

Builds the row of headline numbers at the top of each tab. Takes a list of `(label, value)` tuples and renders each as a pill card.

```
def render_grid(cards: list[str], cols: int = 3) -> None:
    col_objs = st.columns(cols, gap="small")
    for i, card_html in enumerate(cards):
        col_objs[i % cols].markdown(card_html, unsafe_allow_html=True)
```

Distributes cards across `cols` Streamlit columns. `i % cols` cycles through column indices (0, 1, 2, 0, 1, 2, ...) so cards fill left-to-right row by row.

```
competitions = query("""
    WITH match_counts AS (
        SELECT competition_id, season_id, COUNT(*) AS match_count
        FROM matches
        GROUP BY competition_id, season_id
        HAVING COUNT(*) >= 10
    ),
```

First filter: only competitions with at least 10 matches. This removes historical cup finals and other single-game entries in the StatsBomb dataset.

```
    team_balance AS (
        SELECT competition_id, season_id,
               MAX(team_matches) AS max_tm,
               MIN(team_matches) AS min_tm
        FROM (
            SELECT competition_id, season_id, team,
                   COUNT(DISTINCT match_id) AS team_matches
            FROM events
            GROUP BY competition_id, season_id, team
        ) t
        GROUP BY competition_id, season_id
    )
```

Second filter: calculates the most and fewest matches any single team has in each competition. StatsBomb's open data sometimes covers only one "featured" team per competition in full — all other teams then appear only in the fixtures they played against that team, resulting in huge imbalances (e.g. 34 vs 2).

```
    WHERE tb.max_tm::float / NULLIF(tb.min_tm, 0) <= 4
```

Accepts only competitions where the most-covered team has no more than 4× the matches of the least-covered team. This threshold passes balanced leagues (ratio 1.0) and tournaments with knockout stages (ratio up to ~2.3 — a group-stage team vs a finalist) while filtering out the skewed single-team coverages (ratio 17+).

```
comp_labels = {
    f"{c['competition_name']} — {c['season_name']}": c
    for c in competitions
}
```

A dict keyed by the human-readable label string. Used both to populate the selectbox and to look up the full row from the selected label.

```
with st.sidebar:
   selected_label = st.selectbox("Competition & Season", list(comp_labels.keys()), ...)
```

Streamlit re-runs the entire script from top to bottom every time the user changes the selectbox. `selected_label` will hold whatever the user has currently selected.

```
selected = comp_labels[selected_label]
competition_id = selected["competition_id"]
season_id = selected["season_id"]
match_count = selected["match_count"]
```

Unpacks the selected competition's row so the values can be used both in the page header and in the SQL queries below.

```
st.markdown(f"""
<div class="page-header">
  <h1>{comp_name}</h1>
 <p>{season_name}  ·  {match_count} matches  ·  StatsBomb Open Data</p>
</div>
""", unsafe_allow_html=True)
```

The page title. Shows match count so users can immediately judge the data depth for the selected season.

```
tab_teams, tab_players = st.tabs(["Teams", "Players"])
```

Creates two tabs and unpacks their container objects. Everything inside `with tab_teams:` renders only in the Teams tab.

```
with tab_teams:
    teams = query("""
     SELECT rank, team, goals_scored, total_shots, total_passes, shot_conversion,
               goals_pct, shots_pct, passes_pct, conversion_pct
        FROM top_teams
        WHERE competition_id = :cid AND season_id = :sid
        ORDER BY rank
    """, {"cid": competition_id, "sid": season_id})
```

Reads from the `top_teams` materialized view. The `:cid` and `:sid` are SQLAlchemy named parameters, which are safe from SQL injection.

```
        df_teams = pd.DataFrame(teams)
        total_goals = int(df_teams["goals_scored"].sum())
        avg_shots = int(df_teams["total_shots"].mean())
     avg_conv = round(df_teams["shot_conversion"].astype(float).mean() * 100, 1)
```

Computes summary statistics for the headline pills. The `astype(float)` is needed because `shot_conversion` comes back from the database as a `Decimal` type which pandas does not automatically treat as numeric.

```
     search = st.text_input("Search team", placeholder="Filter teams…", key="team_search")
        if search:
            teams = [t for t in teams if search.lower() in t["team"].lower()]
```

Client-side filtering — no database round-trip. The `key=` argument is required to distinguish this widget from the player search box below (Streamlit uses the key to maintain widget state across re-runs).

```
        cards = [team_card(t) for t in teams]
        render_grid(cards, cols=3)
```

Builds all card HTML strings, then passes them to the grid renderer. Each card is a self-contained HTML string injected via `st.markdown`.

```
with tab_players:
    players = query("""
     SELECT rank, player_name, total_goals, total_shots, total_passes, conversion_rate,
               goals_pct, shots_pct, passes_pct, conversion_pct
        FROM top_players
        WHERE competition_id = :cid AND season_id = :sid
        ORDER BY rank
    """, {"cid": competition_id, "sid": season_id})
```

Reads from `top_players`. Limited to 100 rows per competition/season by the view definition.

```
        top_scorer = df_players.iloc[0]["player_name"]
        top_goals  = int(df_players.iloc[0]["total_goals"])
```

The first row is always the top scorer because the view orders by `rank`, which is assigned by goals descending.

---

requirements.txt

```
streamlit       # the web framework and UI runtime
sqlalchemy      # database abstraction layer and connection pooling
psycopg2-binary # PostgreSQL driver (binary wheel, no compilation needed)
requests        # HTTP client used by seed.py to download StatsBomb JSON
python-dotenv   # loads .env file into os.environ
```

.streamlit/config.toml

```
[theme]
base = "light"
```

Sets Streamlit's built-in theme to light mode. This controls the chrome Streamlit generates (scrollbars, tooltips, the sidebar toggle button) that cannot be reached by custom CSS.

```
primaryColor = "#2563eb"
```

The blue used for interactive elements like the active tab underline and focused input borders. Matches the `#2563eb` used in the custom CSS so Streamlit's own components stay consistent.

```
backgroundColor = "#f8fafc"
secondaryBackgroundColor = "#f1f5f9"
textColor = "#0f172a"
```

Slate-based light palette. `backgroundColor` is the main content area, `secondaryBackgroundColor` is used by Streamlit for things like code blocks and expanders.

```
font = "sans serif"
```

Tells Streamlit to use a generic sans-serif stack as the base. The custom CSS then overrides this with Inter specifically.